

Exercise - 5

a. Write a JAVA program give example for “super” keyword

```
class Vehicle {  
    String type = "Generic Vehicle";  
    int maxSpeed = 120;  
    // Superclass constructor  
    Vehicle(String type) {  
        this.type = type;  
        System.out.println("Vehicle constructor called for: " + this.type);  
    }  
    void displayInfo() {  
        System.out.println("Vehicle Type: " + type);  
        System.out.println("Max Speed (Vehicle): " + maxSpeed + " km/h");  
    }  
}  
  
class Car extends Vehicle {  
    String type = "Car"; // Subclass variable with same name as superclass  
    int maxSpeed = 180;  
    // Subclass constructor calling superclass constructor  
    Car(String brand) {  
        super("Automobile"); // Calls Vehicle(String type) constructor  
        System.out.println("Car constructor called for brand: " + brand);  
    }  
    @Override  
    void displayInfo() {  
        // Accessing superclass's instance variable  
        System.out.println("Subclass Type: " + this.type);  
        System.out.println("Superclass Type (using super): " + super.type);  
    }  
}
```

```

        // Accessing superclass's instance variable with same name
        System.out.println("Max Speed (Car): " + this.maxSpeed + " km/h");
        System.out.println("Max Speed (Vehicle using super): " + super.maxSpeed + " km/h");
        // Invoking superclass's method
        super.displayInfo(); // Calls Vehicle's displayInfo()
        System.out.println("Displaying info from Car class.");
    }
}

public class SuperKeywordExample {
    public static void main(String[] args) {
        Car myCar = new Car("Toyota");
        System.out.println("\n--- Calling displayInfo() on Car object ---");
        myCar.displayInfo();
    }
}

```

OUTPUT:

Vehicle constructor called for: Automobile

Car constructor called for brand: Toyota

--- Calling displayInfo () on Car object ---

Subclass Type: Car

Superclass Type (using super): Automobile

Max Speed (Car): 180 km/h

Max Speed (Vehicle using super): 120 km/h

Vehicle Type: Automobile

Max Speed (Vehicle): 120 km/h

Displaying info from Car class.

b. Write a JAVA program to implement Interface. What kind of Inheritance can be achieved?

```
// Define an interface
interface Swimmable {
    void swim();
}

// Define another interface
interface Flyable {
    void fly();
}

// A class implementing multiple interfaces
class Duck implements Swimmable, Flyable {
    @Override
    public void swim() {
        System.out.println("Duck is swimming.");
    }
    @Override
    public void fly() {
        System.out.println("Duck is flying.");
    }
    public void quack() {
        System.out.println("Duck says Quack!");
    }
}

public class InterfaceInheritanceDemo {
    public static void main(String[] args) {
        Duck myDuck = new Duck();
        myDuck.swim(); // Calling method from Swimmable interface
    }
}
```

```
myDuck.fly(); // Calling method from Flyable interface
myDuck.quack(); // Calling method specific to Duck class
// Demonstrating type inheritance

Swimmable anotherSwimmer = new Duck();
anotherSwimmer.swim(); // Can only call Swimmable methods

Flyable anotherFlyer = new Duck();
anotherFlyer.fly(); // Can only call Flyable methods
}
}
```

Output:

Duck is swimming.

Duck is flying.

Duck says Quack!

C. Write a JAVA program that implements Runtime polymorphism.

```
// Superclass
class Animal {
    public void makeSound() {
        System.out.println("The animal makes a sound.");
    }
}

// Subclass 1
class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("The dog barks.");
    }
}

// Subclass 2
class Cat extends Animal {
    @Override
    public void makeSound() {
        System.out.println("The cat meows.");
    }
}

// Main class to demonstrate runtime polymorphism
public class RuntimePolymorphismDemo {
    public static void main(String[] args) {
        // Create a reference of the superclass type
        Animal myAnimal;

        // Assign a Dog object to the superclass reference
```

```
myAnimal = new Dog();  
// Call the overridden method - the Dog's makeSound() is invoked  
myAnimal.makeSound();  
// Assign a Cat object to the superclass reference  
myAnimal = new Cat();  
// Call the overridden method - the Cat's makeSound() is invoked  
myAnimal.makeSound();  
// Assign an Animal object to the superclass reference  
myAnimal = new Animal();  
// Call the original method - the Animal's makeSound() is invoked  
myAnimal.makeSound();  
}  
}
```

Output:

The dog barks.

The cat meows.

The animal makes a sound.